



ЛАБОРАТОРИ АЖИЛ №1 ОНОО-2

Зорилго: Энэ лабораторийн ажлаар AoS (Array of Structures) болон SoA (Structure of Arrays) өгөгдлийн бүтцийг өөрөө бүрэн бичиж, дараах 4 өөр хандалтын аргыг хэмжиж харьцуулна:

Хандалтын дараалал	Хэдэн талбарт хандах вэ
1 Дараалсан (sequential)	Бүх талбар (x, y, z, mass)
2 Дараалсан (sequential)	Зөвхөн нэг талбар (mass)
3 Санамсаргүй (random)	Бүх талбар
4 Санамсаргүй (random)	Зөвхөн нэг талбар

Эдгээр 4 аргыг AoS болон SoA хоёуланд нь хэрэгжүүлснээр, нийт 8 хослол гарна.

Даалгавар 1: Өгөгдлийн бүтэц тодорхойлох

```
// AoS
struct ParticleAoS {
    // TODO: x, y, z, mass — бүгд double төрөлтэй
};

// SoA
struct ParticlesSoA {
    // TODO: x, y, z, mass — бүгд double* (тусдаа массив) төрөлтэй
};
```

Даалгавар 2: Санамсаргүй индекс үүсгэх

```
int *make_random_indices(int n) {
    // TODO: Fisher-Yates алгоритмаар 0..n-1 индексийн санамсаргүй эрэмбэ үүсгэнэ үү
}
```

AoS болон SoA-г харьцуулахдаа ижил санамсаргүй эрэмбийг ашиглана. Индексийг N бүр дээр нэг л удаа үүсгэж, 8 функц бүрд адилхан ашиглана.

Даалгавар 3: 8 ширхэг функц бичих

Функц бүр N элементийг дамжин, тухайн аргын дагуу нийлбэрийг тооцоолж, double утга буцаана (нийлбэр ашиглах шалтгаан: compiler давталтыг "хэрэггүй" гэж үзэж бүрэн арилгахаас сэргийлэх).

AoS дээрх 4 функц

```
// 1. AoS, дараалсан, БҮХ талбар
double sum_aos_seq_all(struct ParticleAoS *p, int n) {
    // TODO: i = 0..n-1 дараалалтайгаар p[i].x + p[i].y + p[i].z + p[i].mass -г
    нийлбэрлэнэ
}
```

```
// 2. AoS, дараалсан, ЗӨВХӨН mass
double sum_aos_seq_one(struct ParticleAoS *p, int n) {
    // TODO: i = 0..n-1 дараалалтайгаар зөвхөн p[i].mass-ийг нийлбэрлэнэ
}

// 3. AoS, санамсаргүй, БҮХ талбар
double sum_aos_rand_all(struct ParticleAoS *p, const int *idx, int n) {
    // TODO: idx[i]-ээр индекслэн, тухайн элементийн БҮХ талбарыг нийлбэрлэнэ
}

// 4. AoS, санамсаргүй, ЗӨВХӨН mass
double sum_aos_rand_one(struct ParticleAoS *p, const int *idx, int n) {
    // TODO: idx[i]-ээр индекслэн, зөвхөн mass-ийг нийлбэрлэнэ
}
```

SoA дээрх 4 функц

```
// 5. SoA, дараалсан, БҮХ талбар
double sum_soa_seq_all(struct ParticlesSoA *p, int n) {
    // TODO: i = 0..n-1 дараалалтайгаар p->x[i]+p->y[i]+p->z[i]+p->mass[i]-г нийлбэрлэнэ
}

// 6. SoA, дараалсан, ЗӨВХӨН mass
double sum_soa_seq_one(struct ParticlesSoA *p, int n) {
    // TODO: зөвхөн p->mass[i]-г нийлбэрлэнэ
}

// 7. SoA, санамсаргүй, БҮХ талбар
double sum_soa_rand_all(struct ParticlesSoA *p, const int *idx, int n) {
    // TODO: idx[i]-ээр индекслэн, БҮХ талбарыг нийлбэрлэнэ
}

// 8. SoA, санамсаргүй, ЗӨВХӨН mass
double sum_soa_rand_one(struct ParticlesSoA *p, const int *idx, int n) {
    // TODO: idx[i]-ээр индекслэн, зөвхөн mass[i]-г нийлбэрлэнэ
}
```

Даалгавар 4: Хугацаа хэмжих механизм

clock() (эсвэл clock_gettime(CLOCK_MONOTONIC, ...) — илүү нарийвчлалтай) ашиглан, функц бүрийн эхлэх/дуусах цагийн зөрүүг секундээр тооцоолно уу.

Зөвлөгөө: Нэг удаагийн хэмжилт эцсийн үнэлгээ болохгүй тул, функц бүрийг 3-5 удаа дуудаж дундажлана уу.

Даалгавар 5: Туршилтын хүрээ

main функц дотор:

1. **3 ялгаатай N** сонгоно уу (жишээ: 1,000,000 / 10,000,000 / 50,000,000 — компьютерийнхээ санах ойн хэмжээнд тохируулж болно)
2. N бүр дээр:
 - AoS болон SoA массивуудыг malloc-аар байршуулж, тогтмол утгаар дүүргэнэ (x=1.0, y=2.0, z=3.0, mass=1.5 гэх мэт)
 - make_random_indices(N)-ээр нэг л удаа санамсаргүй эрэмбэ үүсгэнэ



- 8 функцийг бүгдийг дуудаж, тус бүрийн хугацааг хэмжинэ
- Үр дүнг хэвлэнэ (мөн нийлбэр утгыг хэвлэж зөв эсэхээ шалгаарай)
- free()-ээр санах ойгоо чөлөөлнө

srand(42) гэх мэт тогтмол seed ашиглана уу.

Даалгавар 6: Компайл ба ажиллуулах

```
gcc -O3 your_file.c -o your_program
```

Даалгавар 7: Үр дүнг харуулах

N бүр дээр дараах хүснэгтийг бөглөнө үү (дундаж утгаар):

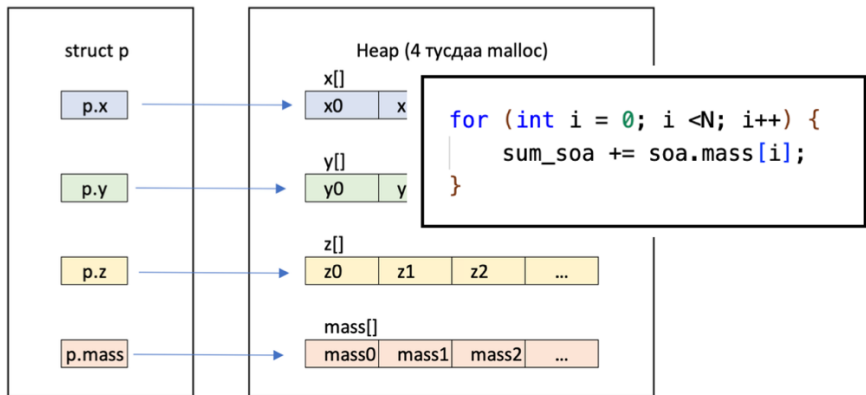
Арга	N=10 ⁶ (сек)	N=10 ⁷ (сек)	N=5×10 ⁷ (сек)
AoS дараалсан, бүх талбар			
AoS дараалсан, зөвхөн mass			
AoS санамсаргүй, бүх талбар			
AoS санамсаргүй, зөвхөн mass			
SoA дараалсан, бүх талбар			
SoA дараалсан, зөвхөн mass			
SoA санамсаргүй, бүх талбар			
SoA санамсаргүй, зөвхөн mass			

Даалгавар 8: Дүн шинжилгээ лаборатори ажлын хамгаалалтанд бэлтгэх.

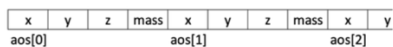
1. AoS дотор "бүх талбар" ба "зөвхөн mass" хандалтыг харьцуулахад ямар ялгаа ажиглагдав (дараалсан хандалтын хувьд)? Энэ ялгааг cache line ашиглалтын ойлголттой холбож тайлбарлана уу.
2. SoA дотор ижил харьцуулалтыг хийхэд (бүх талбар vs зөвхөн mass) ялгаа их байв уу, бага байв уу? Таны онолын таамаглалтай тохирч байна уу?
3. Хамгийн муу гүйцэтгэлтэй хослол аль нь байв? Хамгийн сайн нь? Энэ хоёрын хурдны ялгаа хэдэн дахин байв (N хамгийн их үед)?
4. "Санамсаргүй + зөвхөн нэг талбар" гэсэн хослол дээр AoS болон SoA хоёрын ялгаа их байв уу, бага байв уу? Яагаад ийм үр дүн гарсан гэж бодож байна (санамж: санамсаргүй хандалт үед аль ч тохиолдолд cache line-ийг дахин ашиглах боломж багасдаг)?
5. Хэрэв бодит систем (жишээ: sensor өгөгдөл, тоглоомын физик engine) ашиглах ёстой байсан бөгөөд ихэвчлэн зөвхөн нэг талбар дээр тооцоолол хийдэг байсан бол, AoS болон SoA-ийн аль нэгийг сонгохдоо ямар зүйлийг гол болгон бодох вэ?



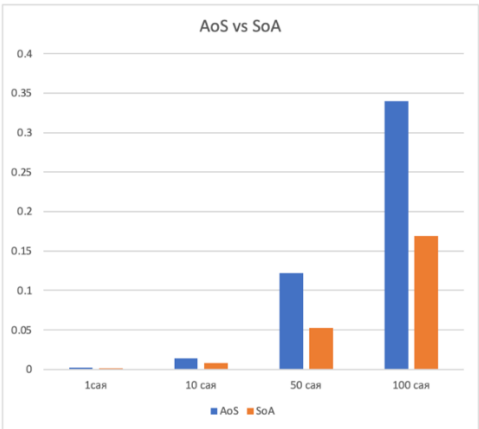
SoA:



AoS



```
for (int i = 0; i < N; i++) {  
    sum_aos += aos[i].mass;  
}
```



N	AoS	SoA
1 сая	0.0027	0.0017
10 сая	0.0141	0.0085
50 сая	0.1218	0.0529
100 сая	0.34	0.1687